

DM—Assignment 1

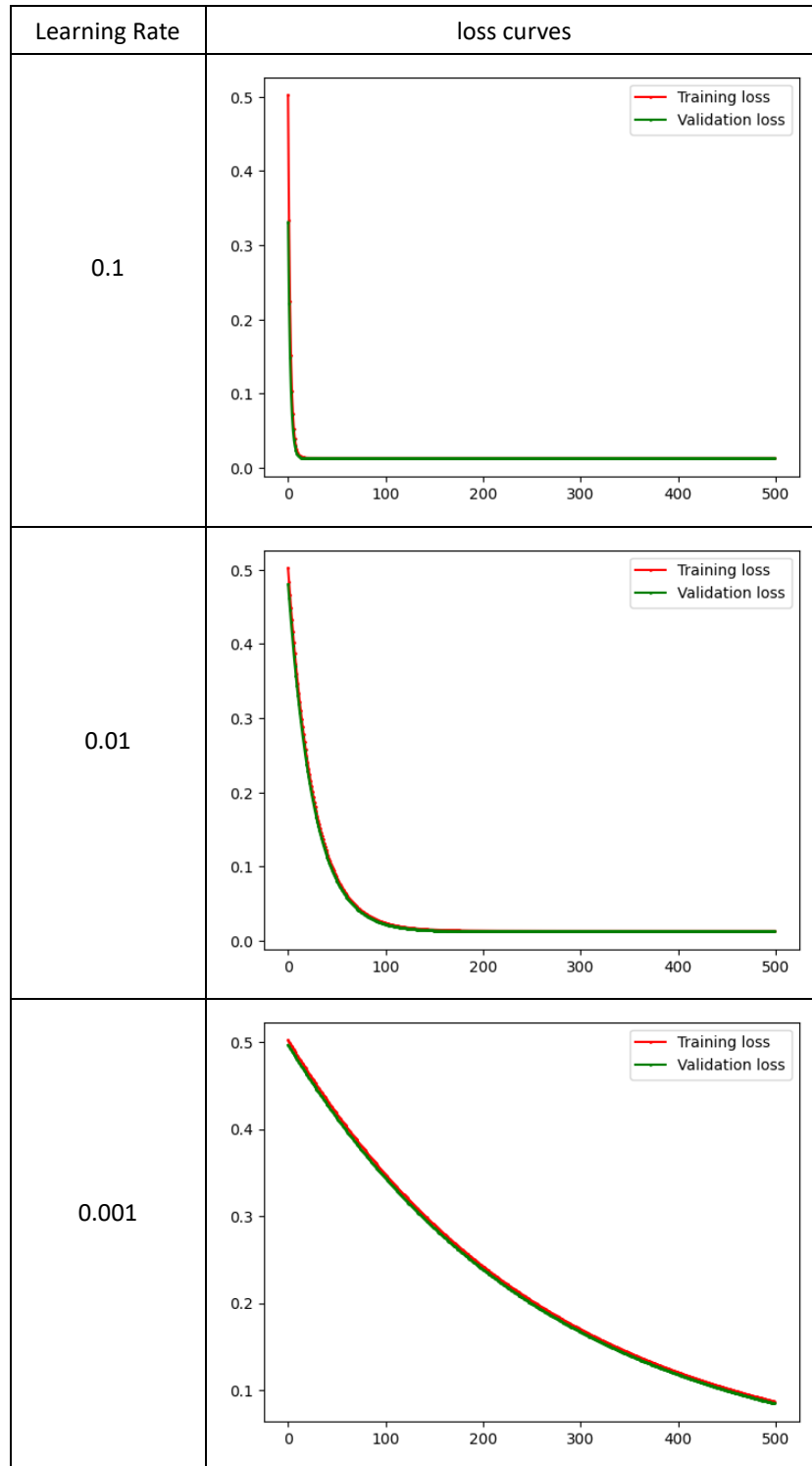
Name：張東浩

Student ID：431551038

GitHub：<https://github.com/doctor-66/535703-Data-Mining-Assignment1>

1. Linear Regression：

1. loss curve in different learning rate



2. evaluation results in different learning rate

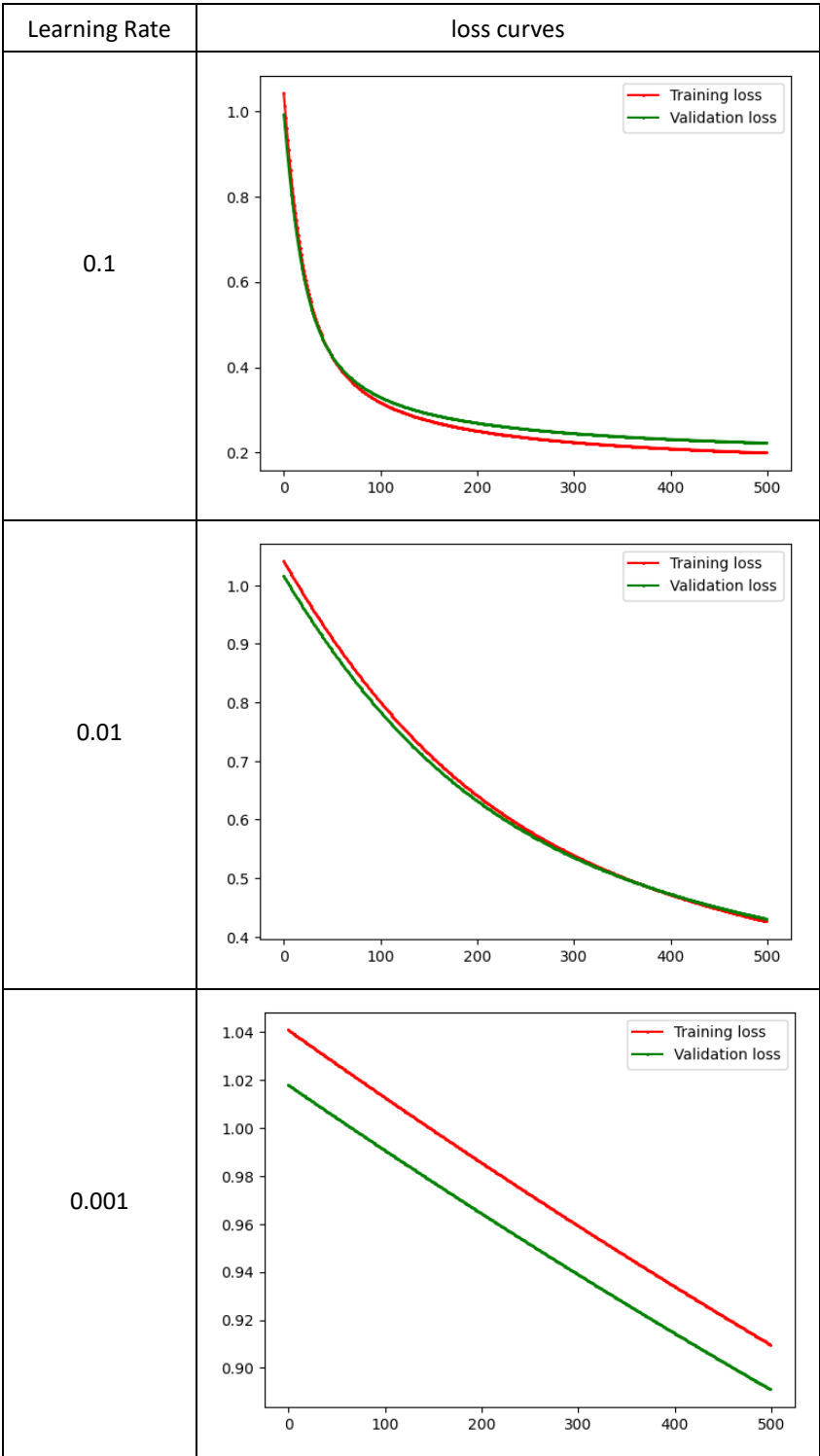
Learning rate	Results
0.1	<pre>=== Linear Regression Evaluation === MSE : 0.0134 MAE : 0.1010 RMSE : 0.1156 R-squared : 0.5690</pre>
0.01	<pre>=== Linear Regression Evaluation === MSE : 0.0134 MAE : 0.1010 RMSE : 0.1156 R-squared : 0.5690</pre>
0.001	<pre>=== Linear Regression Evaluation === MSE : 0.0832 MAE : 0.2369 RMSE : 0.2884 R-squared : -1.6836</pre>

3. Observations

In my observations, the results in Table (1B) are consistent with the trends observed in Table (1A). For learning rates of **0.1** and **0.01**, both training and validation losses decrease smoothly to similarly low levels and remain closely aligned, indicating convergence and generalization. Correspondingly, both setting shows the same performance in Table (1B), with low error values and a positive **R-squared** of **0.5690**. However, **0.1** reaches this solution much faster than **0.01**, demonstrating higher training efficiency. In contrast, for learning rate of **0.001**, the loss decreases much more slowly and remains comparatively high even after 500 epochs in Table (1A), indicating insufficient convergence. These directly reflected in Table (1B), where the model yields substantially higher **MSE**, **MAE**, and **RMSE**, together with a negative **R-squared** (**-1.6836**), suggesting poor predictive performance. Overall, Table (1A) explains the optimization process, and Table (1B) confirms its final outcome. Together, these shows that a learning rate of **0.1** is the most effective choice in this setting because it achieves both rapid convergence and strong final performance.

2. **Logistic Regression :**

(A) loss curve in different learning rate



(B) evaluation results in different learning rate

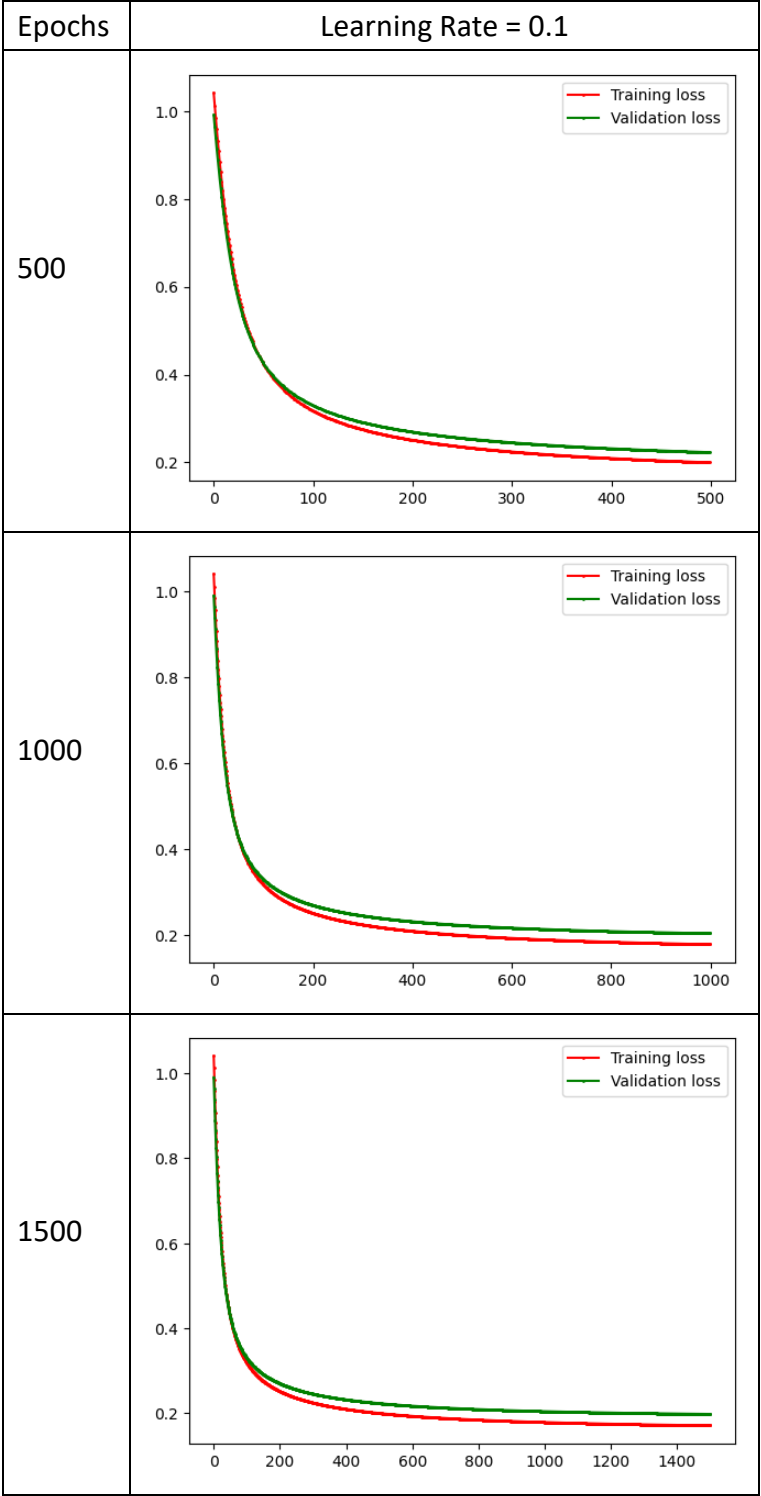
Learning rate	Results
0.1	Logistic Regression Evaluation Accuracy : 0.9050 Precision : 0.9087 Recall : 0.9171 F1-score : 0.9128
0.01	Logistic Regression Evaluation Accuracy : 0.8775 Precision : 0.8784 Recall : 0.8986 F1-score : 0.8884
0.001	Logistic Regression Evaluation Accuracy : 0.3875 Precision : 0.4364 Recall : 0.4424 F1-score : 0.4394

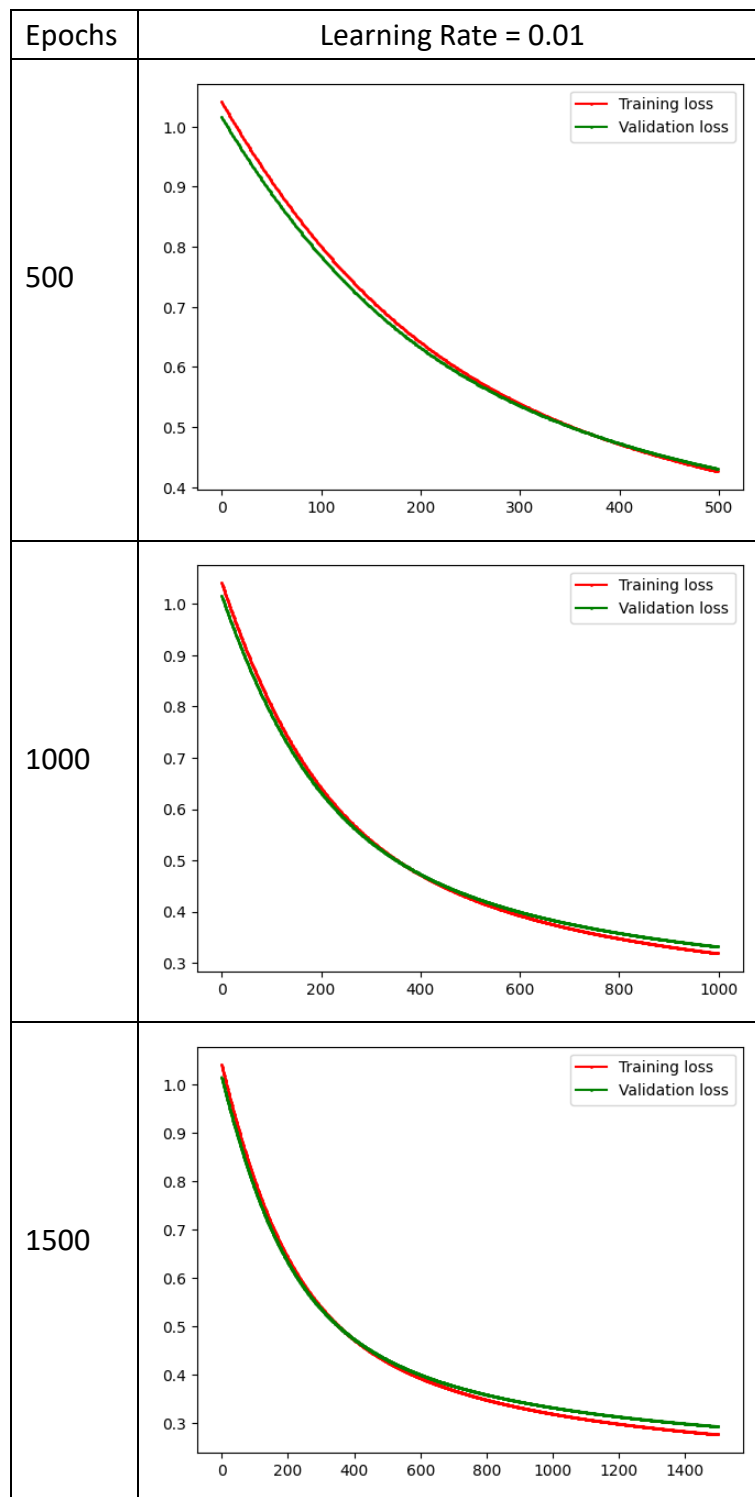
(C) Observations

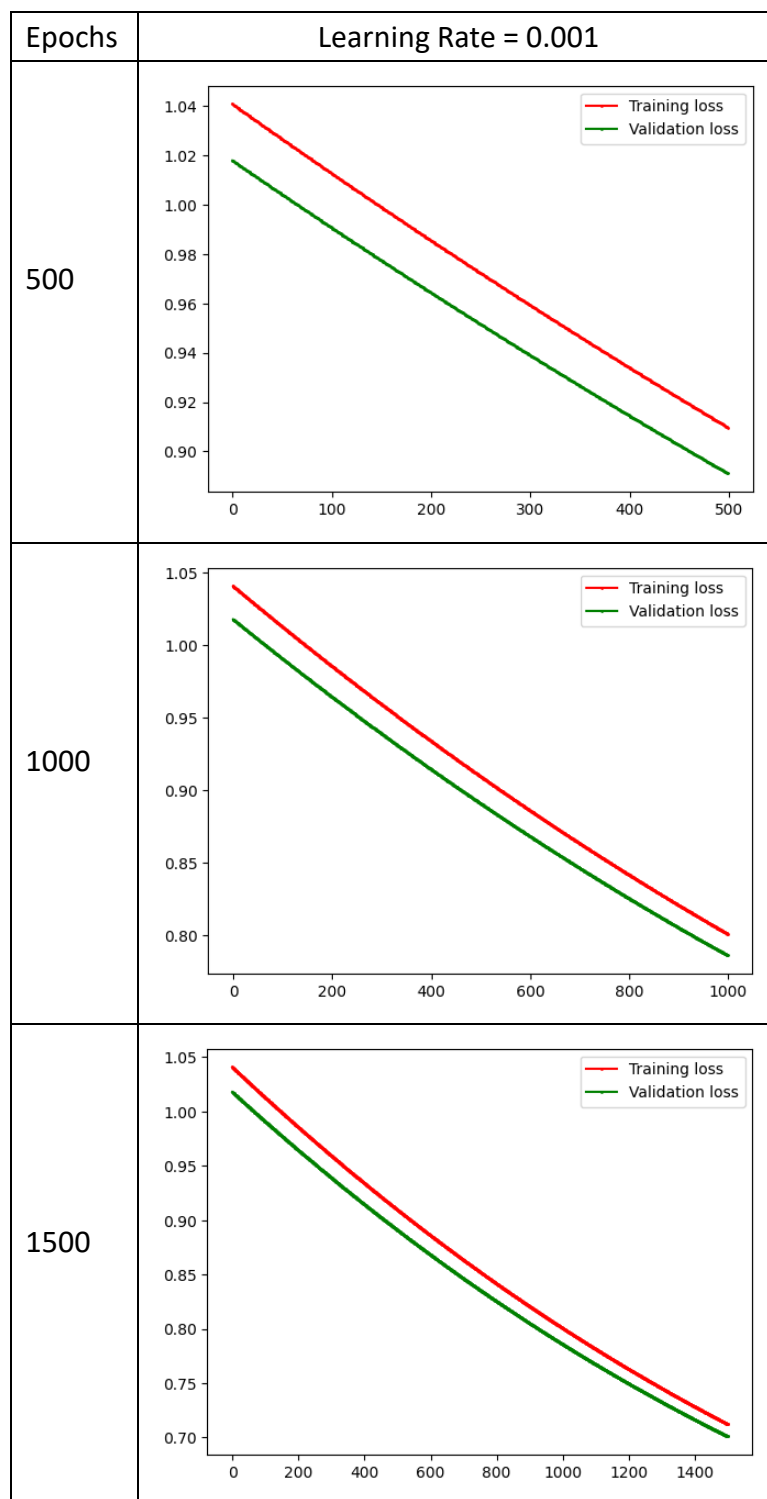
In my observation, the relation between the two table is clearly consistent. With a learning rate of **0.1**, both training and validation losses decrease rapidly and stabilize at the lowest levels among all settings, indicating efficient convergence and generalization. This is reflected in Table (1B), where **0.1** achieves the best overall performance, with **Accuracy = 0.9050**, **Precision = 0.9087**, **Recall = 0.9171**, and **F1-score = 0.9128**. For **0.01**, the loss curves also decrease steadily, but the convergence is noticeably slower and the final loss remains higher than that of **0.1**. Correspondingly, the classification metrics are lower, with **Accuracy = 0.8775**, **Precision = 0.8784**, **Recall = 0.8986**, and **F1-score = 0.8884**, suggesting that the model is able to learn useful decision boundaries but does not reach as favorable an optimum within the same number of epochs. In contrast, when the learning rate is reduced to **0.001**, both training and validation losses remain high throughout training and show only limited reduction, indicating insufficient learning progress under the fixed training budget. This behavior is directly confirmed in Table (B), where the model yields the weakest performance, with **Accuracy = 0.3875**, **Precision = 0.4364**, **Recall = 0.4424**, and **F1-score = 0.4394**. Overall, Table (1A) explains the convergence process, Table (1B) further confirms the final outcomes. Together, demonstrate that **0.1** is the most suitable learning

rate for logistic regression in this setting, as it leads to the fastest convergence and the strongest classification performance.

(D) number of training iteration in different learning rate







(E) evaluation the number of training iteration in different learning rate

Learning rate	Results epochs=500	Results epochs=1000	Results epochs=1500
0.1	Logistic Regression Evaluation Accuracy : 0.9050 Precision : 0.9087 Recall : 0.9171 F1-score : 0.9128	Logistic Regression Evaluation Accuracy : 0.9050 Precision : 0.9087 Recall : 0.9171 F1-score : 0.9128	Logistic Regression Evaluation Accuracy : 0.9050 Precision : 0.9087 Recall : 0.9171 F1-score : 0.9128

0.01	Logistic Regression Evaluation Accuracy : 0.8775 Precision : 0.8784 Recall : 0.8986 F1-score : 0.8884	Logistic Regression Evaluation Accuracy : 0.9025 Precision : 0.9045 Recall : 0.9171 F1-score : 0.9108	Logistic Regression Evaluation Accuracy : 0.9025 Precision : 0.9083 Recall : 0.9124 F1-score : 0.9103
0.001	Logistic Regression Evaluation Accuracy : 0.3875 Precision : 0.4364 Recall : 0.4424 F1-score : 0.4394	Logistic Regression Evaluation Accuracy : 0.4625 Precision : 0.5044 Recall : 0.5253 F1-score : 0.5147	Logistic Regression Evaluation Accuracy : 0.5350 Precision : 0.5683 Recall : 0.5945 F1-score : 0.5811

(F) Observations

For **learning rate = 0.1**, the loss curves in Table (2D) drop rapidly and reach a stable low level within the first 500 epochs, and extending training to 1000 or 1500 epochs produces almost no further visible improvement. This is confirmed by Table (2E), where the evaluation metrics remain exactly the same across all three epoch settings (**Accuracy = 0.9050, Precision = 0.9087, Recall = 0.9171, F1-score = 0.9128**). This indicates that the model has already converged by 500 epochs, and additional iterations do not provide further benefit. For **learning rate = 0.01**, the loss decreases more gradually, and the curves at 1000 and 1500 epochs are clearly lower than those at 500 epochs, indicating that the model continues to optimize when more training iterations are allowed. The evaluation results in Table (2E) reflect this trend: performance improves from **Accuracy = 0.8775** and **F1-score = 0.8884** at 500 epochs to **Accuracy = 0.9025** and **F1-score = 0.9108** at 1000 epochs. However, the improvement from 1000 to 1500 epochs is marginal, and the F1-score slightly decreases to **0.9103**, suggesting that the model has nearly converged by 1000 epochs and that additional training yields diminishing returns. In contrast, for **learning rate = 0.001**, the loss curves remain relatively high even at 1500 epochs and decrease only slowly throughout training, showing that the optimization process is still far from convergence under the given training budget. This is directly reflected in Table (2E), where the evaluation metrics improve progressively as the number of epochs increases, from **Accuracy = 0.3875, F1-score = 0.4394** at 500 epochs, to **Accuracy = 0.4625, F1-score = 0.5147** at 1000 epochs, and further to **Accuracy = 0.5350, F1-score = 0.5811** at 1500 epochs. Nevertheless, these results remain substantially worse than those obtained with larger learning rates, indicating that the update step is too small to achieve competitive performance efficiently. Overall, Table (2D) explains how convergence speed differs across learning rates and epoch settings, while Table (2E) confirms how these optimization differences translate into final classification performance. Taken together, the results show that **0.1** is the most effective learning rate in this setting because it reaches the best performance within only 500 epochs, whereas **0.01** requires more iterations to approach a similar level, and **0.001** remains under-optimized even after 1500 epochs.

3. Data Preprocessing :

(A) median and Standard deviation for column with miss value

For the column that having miss value, they are "SepalLengthCm", "SepalWidthCm", "PetalLengthCm", "PetalWidthCm" and "BranchLength". For "SepalLengthCm" **median = 6.3000** and **std = 1.0371**. For "SepalWidthCm" **median = 2.9000** and **std = 0.3896**. For "PetalLengthCm" **median = 5.0856** and **std = 1.5828**. For "BranchLength" **median = 1.6000** and **std = 0.7067**. For "BranchLength" **median = 16.3000** and **std = 1.0352**.

```
Columns with missing values before imputation:  
SepalLengthCm: median = 6.3000, std = 1.0371  
SepalWidthCm: median = 2.9000, std = 0.3896  
PetalLengthCm: median = 5.0856, std = 1.5828  
PetalWidthCm: median = 1.6000, std = 0.7067  
BranchLength: median = 16.3000, std = 1.0352
```

(B) replace the missing values using "Nearest Neighbors Imputation"

For the column after filling the missing value by **KNN(N = 5)**, "SepalLengthCm" **median = 6.3000** and **std = 1.0093**. For "SepalWidthCm" **median = 2.9000** and **std = 0.3724**. For "PetalLengthCm" **median = 5.0357** and **std = 1.5150**. For "BranchLength" **median = 1.7000** and **std = 0.6946**. For "BranchLength" **median = 16.3000** and **std = 1.0110**.

```
Columns with missing values after KNN imputation:  
SepalLengthCm: median = 6.3000, std = 1.0093  
SepalWidthCm: median = 2.9000, std = 0.3724  
PetalLengthCm: median = 5.0357, std = 1.5150  
PetalWidthCm: median = 1.7000, std = 0.6946  
BranchLength: median = 16.3000, std = 1.0110
```

In my observations, similar pattern can be observed in the **standard deviation**. After KNN imputation, the standard deviation of all affected columns becomes slightly smaller: **SepalLengthCm** decreases from **1.0371** to **1.0093**, **SepalWidthCm** from **0.3896** to **0.3724**, **PetalLengthCm** from **1.5828** to **1.5150**, **PetalWidthCm** from **0.7067** to **0.6946**, and **BranchLength** from **1.0352** to **1.0110**. This suggests that KNN imputation slightly reduces data variability, which is expected because missing values are replaced by estimates derived from neighboring observations, making the completed data somewhat smoother. Overall, the comparison shows that **KNN imputation preserves the general distribution of the dataset while modestly reducing dispersion**, indicating that it is an effective method for handling missing values without causing major statistical distortion.

Additional experiments with different KNN neighborhood sizes (**N = 3, 5, 7, 9**)

show that the imputation results remain generally stable, indicating that the dataset is not highly sensitive to moderate changes in the number of neighbors. Across all settings, the medians and standard deviations of the affected columns vary only slightly, which suggests that KNN imputation preserves the overall distribution of the data regardless of the exact value of **N**. However, as **N** increases, some features, such as **SepalWidthCm** and **PetalLengthCm**, show a small downward shift in median and a slight reduction in standard deviation, implying that larger neighborhoods produce smoother and more averaged imputations. In contrast, smaller **N** values rely more heavily on closer local samples and therefore retain slightly more local variation. Overall, the comparison suggests that changing **N** mainly affects the degree of smoothing, while the completed data remain statistically consistent across these settings.

```
Columns with missing values after KNN (N=3) imputation:  
SepalLengthCm: median = 6.3000, std = 1.0106  
SepalWidthCm: median = 2.9000, std = 0.3737  
PetalLengthCm: median = 5.0595, std = 1.5791  
PetalWidthCm: median = 1.7000, std = 0.6951  
BranchLength: median = 16.3000, std = 1.0126
```

```
Columns with missing values after KNN (N=7) imputation:  
SepalLengthCm: median = 6.3000, std = 1.0089  
SepalWidthCm: median = 2.8571, std = 0.3722  
PetalLengthCm: median = 5.0041, std = 1.4669  
PetalWidthCm: median = 1.6929, std = 0.6942  
BranchLength: median = 16.3000, std = 1.0110
```

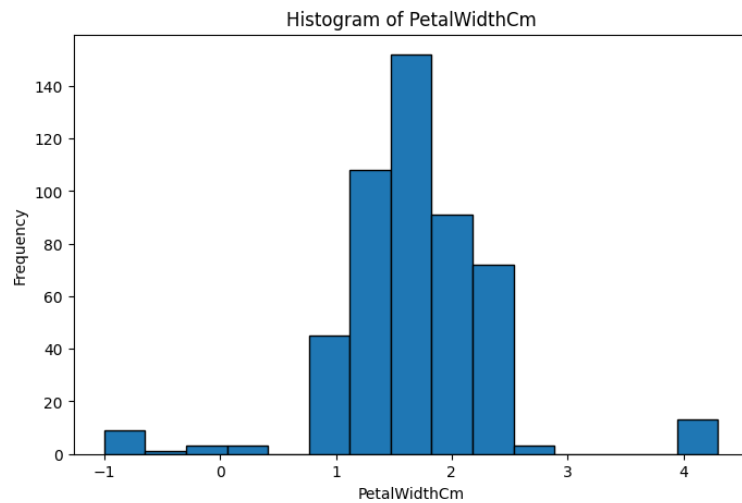
```
Columns with missing values after KNN (N=9) imputation:  
SepalLengthCm: median = 6.3000, std = 1.0086  
SepalWidthCm: median = 2.8722, std = 0.3725  
PetalLengthCm: median = 5.0055, std = 1.4388  
PetalWidthCm: median = 1.6500, std = 0.6942  
BranchLength: median = 16.3000, std = 1.0108
```

4. Data Exploration :

(A) histogram of the column "PetalWidthCm" in "NYCU_Iris.csv"

The histogram shows that **PetalWidthCm** is mainly concentrated between approximately **1.0 and 2.5**, with the highest frequency between **1 to 2**. Indicating that most samples fall within a relatively narrow central range. The distribution is **not perfectly symmetric**, and it appears to be **slightly right-skewed**, as there are a few larger values extending toward the right side. In addition, the histogram contains a small number of values far from the main cluster, including some unusually low values near or below **0** and one very large value above **4**, which may be considered potential outliers or abnormal

observations. Overall, the feature distribution suggests that most observations are grouped around the middle range, while a limited number of extreme values are present at both ends.



(B) report the largest positive correlation with the “PetalWidthCm” column
The largest Pearson correlation coefficient with “PetalWidthCm” that is 0.9917 from “PetalWidthCompactness”.

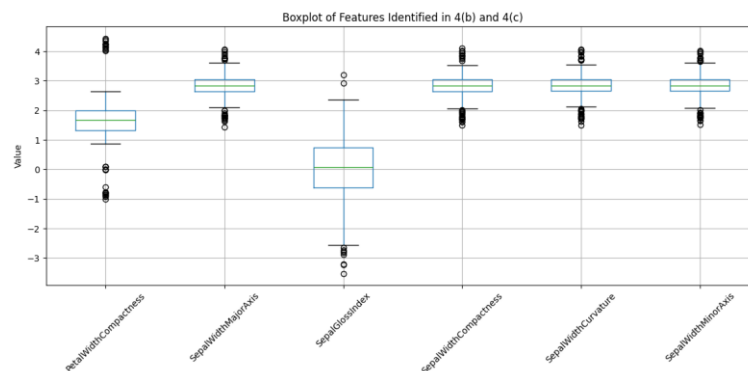
```
Feature with the largest positive correlation with PetalWidthCm:
PetalWidthCompactness: 0.9917
```

(C) top 5 features with the strongest negative correlations with the 'PetalWidthCm' column.

The top 5 features that having negative correlation with “PetalWidthCm” they are, **“SepalWidthMajorAxis” = -0.096350**. **“SepalGlossIndex” = -0.095200**, **“SepalWidthCompactness” = -0.088496**, **“SepalWidthCurvature” = -0.081334** and **“SepalWidthMinorAxis” = -0.074392**.

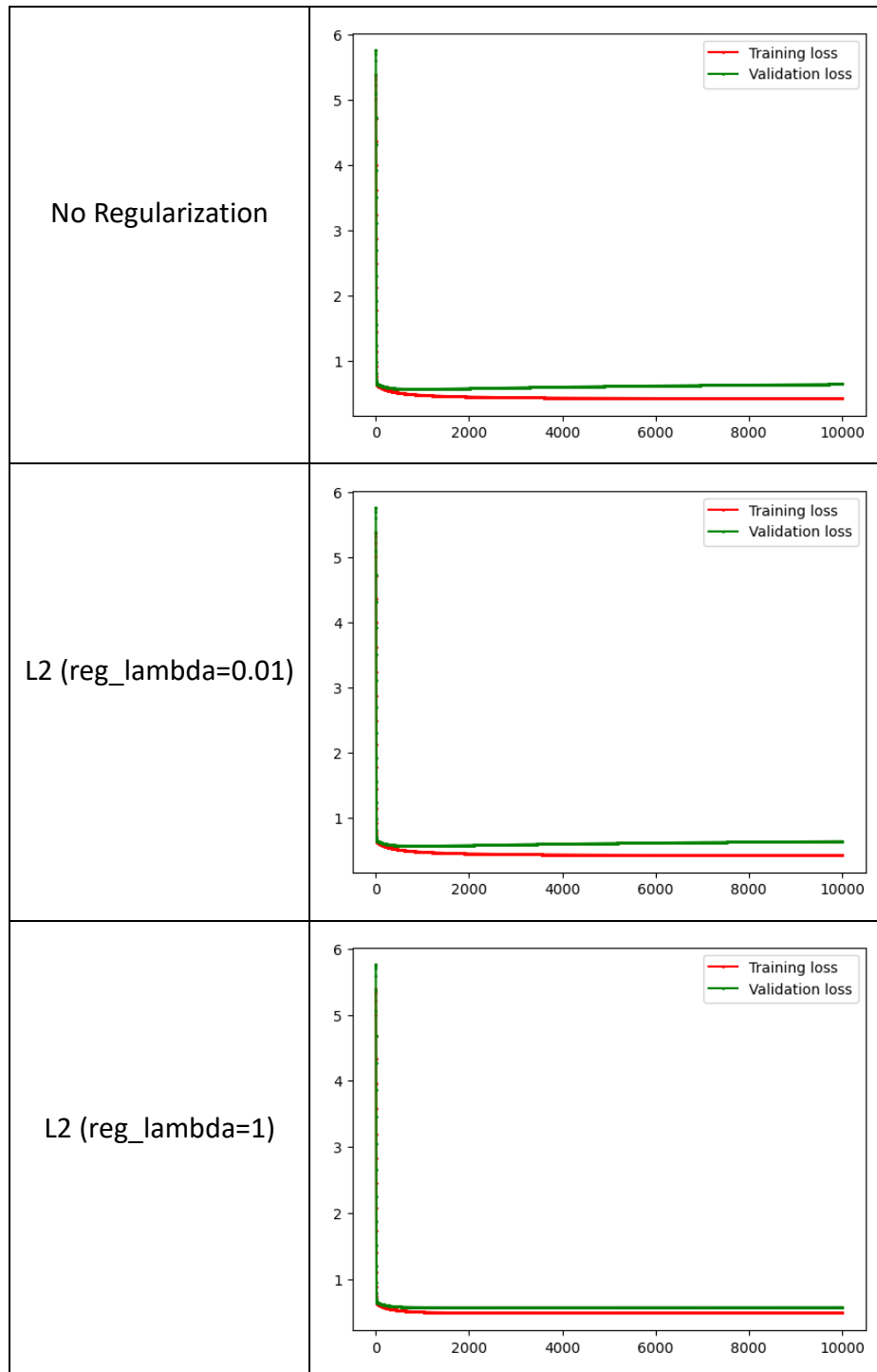
```
Top 5 features with the strongest negative correlations with PetalWidthCm:
SepalWidthMajorAxis    -0.096350
SepalGlossIndex        -0.095200
SepalWidthCompactness  -0.088496
SepalWidthCurvature    -0.081334
SepalWidthMinorAxis    -0.074392
```

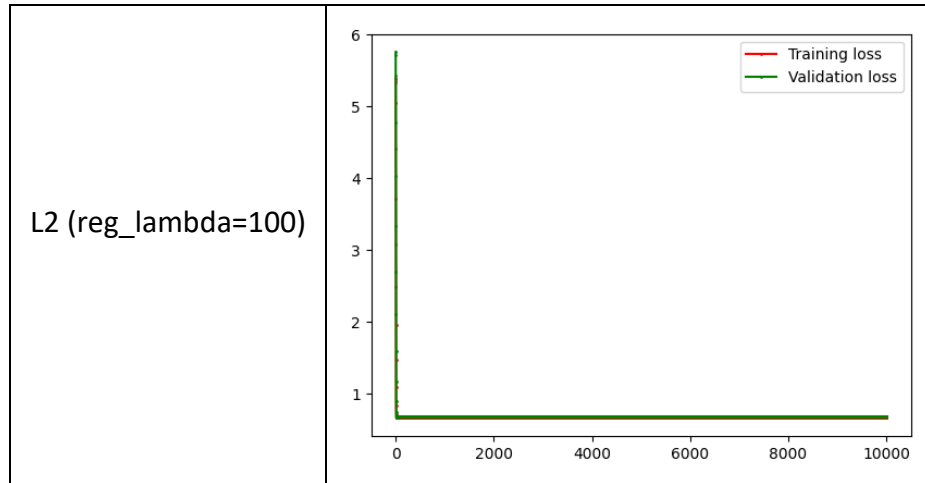
(D) boxplot for all features found in 4(b) and 4(c)



5. Regularization :

(A) L2 Regularization





(B) evaluation the number of training iteration in different learning rate

No Regularization	L2(reg_lambda=0.01)	L2 (reg_lambda=1)	L2(reg_lambda=100)
Model Evaluation Accuracy : 0.7200 Precision : 0.7125 Recall : 0.7500 F1-score : 0.7308	Model Evaluation Accuracy : 0.7200 Precision : 0.7125 Recall : 0.7500 F1-score : 0.7308	Model Evaluation Accuracy : 0.7400 Precision : 0.7342 Recall : 0.7632 F1-score : 0.7484	Model Evaluation Accuracy : 0.5067 Precision : 0.5067 Recall : 1.0000 F1-score : 0.6726

(C) Observations

In Table (5A), all settings show a rapid decrease in both training and validation loss at the beginning of training, indicating that the model can fit the data efficiently regardless of the regularization strength. However, the relationship between the training and validation curves differs across settings, and this difference is reflected in the final metrics reported in Table (5B). With **no regularization** and **L2 regularization with $\lambda = 0.01$** , the loss curves are almost identical, and the final evaluation results are exactly the same (**Accuracy = 0.7200, Precision = 0.7125, Recall = 0.7500, F1-score = 0.7308**). This indicates that such a small regularization strength has little practical effect on the model behavior or predictive performance. When the regularization strength is increased to **$\lambda = 1$** , the training and validation losses remain close and stable while the final classification metrics improve to **Accuracy = 0.7400, Precision = 0.7342, Recall = 0.7632, and F1-score = 0.7484**, suggesting that moderate L2 regularization improves generalization by constraining the model parameters without excessively limiting learning capacity. In contrast, when the regularization becomes too strong (**$\lambda = 100$**), the training and validation losses quickly flatten at a relatively higher level and almost overlap completely, indicating that the model is overly constrained and unable to learn sufficiently rich decision boundaries. This behavior is directly confirmed by Table (5B), where the performance drops

substantially to **Accuracy = 0.5067** and **F1-score = 0.6726**. Although the recall reaches **1.0000**, the precision falls to **0.5067**, implying that the model predicts the positive class too aggressively and sacrifices overall discriminative ability. Overall, Table (5A) explains how different levels of L2 regularization affect the optimization dynamics, while Table (5B) confirms their impact on final performance. Taken together, the results show that **moderate regularization ($\lambda = 1$)** provides the best balance between fitting ability and generalization, whereas **too little regularization** offers no clear benefit and **too much regularization** leads to underfitting.